COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - Data Structures and Algorithms**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**10-07-26**	**DATE SUBMITTED:**	**10-09-26**
NAME:	**Christian Lee C. Bandigan Cenndy M. Nieles**	**INSTRUCTOR:**	**Engr. Jimlord M. Quejado**

ACTIVITY NO.11 **Hands-on Activity 11.1 Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

Assess if two arrays have any common values.

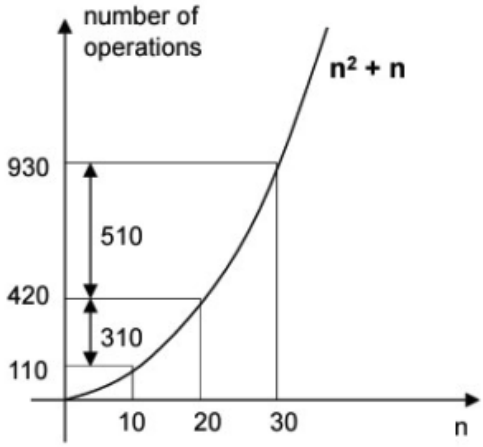
```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
          return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

Let m = the length of array x.
Let n = the length of array y.
The loop in diff repeats n times.
Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **_m * n_**
Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
Bandigan

- The graph shows the worst case of the algorithm which is $O(n^2)$ which is a quadratic function. So, the number of operations increases quadratically as the array size increases.

Nieles:
-The array size increases and the number of operations also increases.The algorithm has a worst case time complexity of O(n^2) .

ILO B : Analyze the results of runtime performance by comparing experimental and theoretical analysis

Code	

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {
    }

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
```

```cpp
    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

// Function prototype (needed because search_test calls it before it is defined)
bool needs_vaccination(Student P, std::vector<Student>& people);

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count() << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") " << "does not need vaccination." << std::endl;
}
```

```cpp

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

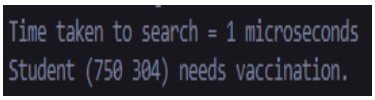
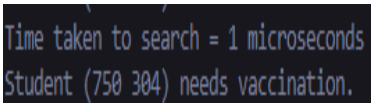
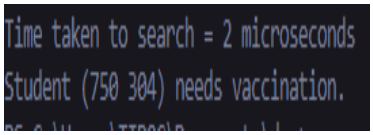
Input Size (N)	Screenshot	Observation(s)
1000		It only took 1 microsecond because it uses a binary search.
10000		The list became 10 times larger but the execution speed stays the same because binary search halves the search range on every step.
100000		It only increased to 2 microseconds even though the list was 100 times larger than the first test. Binary search is still efficient even for large datasets (O(logn)).

Table 11-1.

Analysis:

Bandigan

- The experimental analysis results shows that the execution time mostly stayed the same at 1-2 microseconds range even though the input size increased from 1000 students to 100000 students. This makes sense because binary search has a time complexity of O(logn) meaning it cuts the dataset in half every iteration.

Nieles

The results show that the binary search remains efficient even when the list size increases.The search only increases slightly because the binary search halves the search range at each step.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Theoretical Analysis

```
bool Unique(int A[], int n){
  for(int i = 0; i<n-1;i++){
    for(int j = i+1; j<n; j++){
      if(A[i] == A[j]){
        return false;
      }
    }
  }
  return true;
}
```

$T(n) = (n^2-n) / 2$

Code

supplementary > problem1 > prob1.cpp > ...

```
#include <iostream>
#include <chrono>

bool Unique(int A[], int size);
void Experimental(int size);

int main(){
  Experimental(1000);
  Experimental(10000);
  Experimental(100000);

  return 0;
}

bool Unique(int A[], int n){
  for(int i = 0; i<n-1;i++){
    for(int j = i+1; j<n; j++){
      if(A[i] == A[j]){
        return false;
      }
    }
  }
  return true;
}

void Experimental(int size){
  int* arr = new int[size];

  for(int i = 0; i < size;i++){
    arr[i] = i;
  }
  auto start = std::chrono::high_resolution_clock::now();
  std::cout << "Array is Unique?: " << Unique(arr,size)<< std::endl;

  auto stop = std::chrono::high_resolution_clock::now();
  auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop-start).count();
  std::cout << "Runtime: " << duration << " milliseconds\n";
}
```

Output	PS C:\Users\TIPQC\Documents\dsa\supplementary\problem1> Array is Unique?: 1 Runtime: 3 milliseconds Array is Unique?: 1 Runtime: 170 milliseconds Array is Unique?: 1 Runtime: 12766 milliseconds

Analysis and Comparison

Bandigan

- Since the theoretical analysis of the program is T(n) = (n^2 -n)/2 its worst time complexity is O(n^2). This reflects in the output of the experimental analysis where if you increase the number of elements in the array the runtime quadruples.

Nieles

- The program uses the nested loops to compare each element with other elements in the array , and if two elements are equal it turns to false; otherwise it will go to true when all elements are unique. The output shows that the array is unique in all three tests, with execution times of 3 milliseconds .

Problem 2

Theoretical Analysis

```
4   long long callCount = 0;
5
6   long long rpower(long long x, int n) {
7       callCount++;
8       if (n == 0) return 1;
9       return x * rpower(x, n - 1);
10  }
11
12  long long brpower(long long x, int n) {
13      callCount++;
14      if (n == 0) return 1;
15      if (n % 2 == 1) {
16          long long y = brpower(x, (n - 1) / 2);
17          return x * y * y;
18      } else {
19          long long y = brpower(x, n / 2);
20          return y * y;
21      }
22  }
```

T(n)=T(n-1)+0(1)

Code

```cpp
#include <iostream>
#include <chrono>

long long callCount = 0;

long long rpower(long long x, int n) {
    callCount++;
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

long long brpower(long long x, int n) {
    callCount++;
    if (n == 0) return 1;
    if (n % 2 == 1) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        long long y = brpower(x, n / 2);
        return y * y;
    }
}
```

```cpp
int main() {
    for (int n : {8, 16, 32, 64, 128, 256, 512, 1024}) {
        callCount = 0;

        auto b = std::chrono::steady_clock::now();
        rpower(1, n);
        auto e = std::chrono::steady_clock::now();
        long long rc = callCount;
        auto rt = std::chrono::duration_cast<std::chrono::nanoseconds>(e - b).count();
```

```cpp
        callCount = 0;
        b = std::chrono::steady_clock::now();
        brpower(1, n);
        e = std::chrono::steady_clock::now();
        auto bt = std::chrono::duration_cast<std::chrono::nanoseconds>(e - b).count();

        std::cout << "n = " << n << " | rpower: calls = " << rc << ", " << rt  << " ns | brpower: calls = " << callCount <<
        ", " << bt << " ns" << std::endl;
    }

    return 0;
}
```

Output

```
n = 8 | rpower: calls = 9, 200 ns | brpower: calls = 5, 200 ns
n = 16 | rpower: calls = 17, 100 ns | brpower: calls = 6, 200 ns
n = 32 | rpower: calls = 33, 300 ns | brpower: calls = 7, 200 ns
n = 64 | rpower: calls = 65, 400 ns | brpower: calls = 8, 200 ns
n = 128 | rpower: calls = 129, 900 ns | brpower: calls = 9, 200 ns
n = 256 | rpower: calls = 257, 1200 ns | brpower: calls = 10, 200 ns
n = 512 | rpower: calls = 513, 16900 ns | brpower: calls = 11, 200 ns
n = 1024 | rpower: calls = 1025, 26900 ns | brpower: calls = 12, 200 ns
```

Analysis and Comparison

Nieles

Therefore the results show the rpower() has O(n) time complexity while bpower () has O(log n) time complexity ,

making the bpower() more efficient for a larger input value based on the display number of calls and execution time that shows in output.

Conclusion

Bandigan

- In this activity I learned how to do basic algorithm analysis to know the runtime of different programs. I was able to know the runtime of different programs using both theoretical analysis and experimental analysis. Moreover, in the supplementary activity, I was able to create a program to solve problems, analyze each program to know the runtime using theoretical and experimental analysis, and compare them to know if they are correct. Knowing the runtime of your programs is very important to make your code efficient even if you have hardware constraints.

Nieles

- In this activity about Basic Algorithm Analysis how to analyze and compare the performance of different algorithms and know the different theoretical analysis and experimental analysis and the functions calls, and execution time, and time complexity. And observed that the basic recursive power function requires more calls as the input values increase while the binary recursive power functions perform fewer calls. I learned how each function works,how recursive calls affects execution time, and how to determine their complexity and also help me know the how to analyze an algorithm also helps me to identify which method is more efficient based on the number of each function.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.